

Topic Modelling and Sentiment Analysis

Social and Cultural Analytics (7AAVBCS5)

Dr Mark J. Hill · mark.j.hill@kcl.ac.uk

2026-12-07



Lecture Plan

- **Part I: Word co-occurrence**
 - Bag of words, document–term matrix
- **Part II: Topic modelling**
 - Latent Dirichlet Allocation
 - Choosing the number of topics
- **Part III: Sentiment analysis**
 - Lexicon-based scoring
 - Interpretation and limitations
- **Part IV: Live coding**

Key Learning Outcomes

By the end of this week you will be able to:

- Explain what topic modelling is and where it is useful
- Construct a document–term matrix
- Run LDA and interpret topic outputs
- Apply lexicon-based sentiment analysis in

Python

- Interpret sentiment trends across documents
- **Say what these methods cannot tell you**

Review

Everything from types and variables through data frames, missing data, categories, tables, visualisation, correlation, functions and loops, to the NLP pipeline.

This week: two methods that find structure you did not specify in advance — and the limits of both.

Part I: Word Co-occurrence

Bag of words and the document–term matrix

Bag of words

A representation of text that ignores grammar and word order. Simply a record of which words appear and how often.

Lets you compare documents by shared vocabulary — which terms co-occur, and how often.

Document–term matrix (DTM)

The bag of words as a table.

- **Rows** = documents
- **Columns** = terms
- **Cells** = how often each term appears in each document

“Bag of words” is not a figure of speech. Word order is genuinely discarded: *dog bites man* and *man bites dog* are identical.

Word co-occurrence

Tracking which words appear together in the same document, sentence, or window of text.

If two words repeatedly appear near each other across many texts, we infer they are related in meaning or usage.

Co-occurrence is **purely statistical** — based on patterns in data, not on linguistic or syntactic rules.

“You shall know a word by the company it keeps.”

(Firth)

Building a DTM

```
1 from pathlib import Path
2
3 from sklearn.feature_extraction.text import CountVectorizer
4
5 files = sorted(Path("../data/SUA").glob("*.txt"))
6 docs, names = [], []
7 for path in files:
8     text = path.read_text(encoding="utf-8", errors="replace")
9     if text.strip(): # skip the empty 1790 file
10         docs.append(text)
11         names.append(path.stem)
12
13 vec = CountVectorizer(stop_words="english", min_df=10, max_df=0.9)
14 dtm = vec.fit_transform(docs)
15 print(f"{dtm.shape[0]} documents x {dtm.shape[1]} terms")
16 print(f"sparsity: {100 * (1 - dtm.nnz / (dtm.shape[0] * dtm.shape[1])):.1f}%")
```

```
227 documents x 7096 terms
sparsity: 81.6%
```

Pre-processing decisions

Why remove stop words?

They do not define topics, and removing them usually increases clarity.

Caveat: it distorts stylistic analysis. Keep them for authorship or syntax work.

Stemming vs lemmatising

R's `tm` stemmed: `govern`,
`governs`, `governing` →
`govern`, and `continuation` →
`continu`.

scikit-learn does **not** stem.

Use spaCy lemmas from last week instead — slower, but it produces real words.

Stemming reduces vocabulary and can improve coherence. It also produces non-words, which makes topics harder to read.

Part II: Topic Modelling

What is topic modelling?

We often have corpora that are:

- Too large to read manually
- Too heterogeneous to label by hand
- Too complex to summarise with frequency counts

Typical applications: news articles, social media, government documents, speeches, diaries, interviews, literature

Topic modelling lets us:

- Identify latent themes
- Explore structure in large corpora
- Compare documents by thematic content
- Discover patterns we did not pre-define

What it actually does

Detects clusters of words that frequently occur together.

If certain words often co-occur — *goal, match, manager, transfer* — the model treats that cluster as a topic.

Documents are then expressed as **proportions** of those discovered topics.

What a topic is not

- Not a label
- Not a “true category”
- Not an interpretation

It is a **probability distribution over words**.

What a document is

A mixture of topics, in different proportions.

Latent Dirichlet Allocation

LDA assumes

- Similar topics use similar words
- Texts are bags of words
- Every document is a mixture of topics
- Every topic is a mixture of words

Key properties

- **Unsupervised** — you do not tell it the themes
- **Probabilistic**
- **Non-deterministic** — different runs give different answers

Interpretation is key.

Blei et al., “Latent Dirichlet Allocation”, *JMLR* 3 (2003) 993–1022

Running LDA

```
1 from sklearn.decomposition import LatentDirichletAllocation
2
3 lda = LatentDirichletAllocation(n_components=5, random_state=0, max_iter=10)
4 lda.fit(dtm)
5
6 terms = vec.get_feature_names_out()
7 for k, component in enumerate(lda.components_):
8     top = [terms[i] for i in component.argsort()[-8:][::-1]]
9     print(f"topic {k}: " + ", ".join(top))
```

```
topic 0: 000, general, service, department, law, secretary, foreign, fiscal
topic 1: shall, present, subject, mexico, constitution, general, treaty, necessary
topic 2: law, 000, men, present, business, work, shall, service
topic 3: 000, present, treasury, debt, general, condition, duties, currency
topic 4: world, america, work, americans, security, help, federal, program
```

Reading the topics

Look at the output. Three things worth noticing:

- **000 appears in three topics.** Numbers survived pre-processing, and they are not a theme
- Several topics share `general`, `present`, `shall` — high-frequency political boilerplate
- Topic 4 (`world`, `america`, `security`, `program`) is recognisably **modern**; topic 3 (`treasury`, `debt`, `currency`) is recognisably **nineteenth-century fiscal**

The model found the difference between eras, not between subjects. That is a real finding — and it is not the finding you were looking for.

How many topics?

There is no correct answer. Approaches:

- **Interpretability** — do the topics make sense to a domain expert? Still the standard in practice
- **Perplexity** — how surprised the model is by held-out data. Available in scikit-learn, and often disagrees with human judgement
- **Coherence** — how often the top words of a topic co-occur. R's `topicdoc` provided this; scikit-learn does not

Documents as mixtures

```
1 import pandas as pd
2
3 doc_topics = pd.DataFrame(lda.transform(dtm),
4                             index=names,
5                             columns=[f"topic_{i}" for i in range(5)]).round(3)
6 doc_topics.loc[["Washington_1791", "Roosevelt_1944", "Obama_2016"]]
```

	topic_0	topic_1	topic_2	topic_3	topic_4
Washington_1791	0.000	0.994	0.000	0.005	0.000
Roosevelt_1944	0.022	0.000	0.413	0.000	0.564
Obama_2016	0.000	0.000	0.000	0.000	1.000

Part III: Sentiment Analysis

Sentiment analysis

“Deals with the problem of deciding whether a piece of text, e.g. a sentence or a phrase, is expressing some particular sentiment, e.g. positivity or negativity”
(Castellucci 2015)

Seeks to

- Detect affective tone
- Classify text as positive, negative, neutral
- Identify emotional categories

Applications: reviews, political communication, social media, public opinion

Challenges

- Sentiment is context-dependent
- Sarcasm, irony, **negation**
- Domain-specific vocabulary
- Mixed sentiment within a sentence
- Short texts behave differently

- Lexicons may not match the domain

Lexicon-based sentiment

```
1 from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
2
3 analyser = SentimentIntensityAnalyzer()
4 for sentence in ["This is a wonderful and hopeful day.",
5                  "I am happy", "I am not happy"]:
6     print(f"{sentence:38} {analyser.polarity_scores(sentence)['compound']:+.4f}")
```

This is a wonderful and hopeful day.	+0.7906
I am happy	+0.5719
I am not happy	-0.4585

The trap: VADER on long documents

```
1 speech = Path("../data/SUA/Adams_1797.txt").read_text(encoding="utf-8", errors="replace")
2 print("whole speech compound:", analyser.polarity_scores(speech)["compound"])
```

```
whole speech compound: 0.9996
```

Every speech scores about **0.9999**. The number is real, reproducible, and meaningless.

Scoring sentences, then aggregating

```
1 import re
2
3 sentences = [s for s in re.split(r"(?<=[.!?])\s+", speech) if len(s.split()) > 3]
4 scores = [analyser.polarity_scores(s)["compound"] for s in sentences[:400]]
5 print(f"{len(scores)} sentences, mean compound {sum(scores) / len(scores):+.3f}")
```

```
59 sentences, mean compound +0.236
```

Unit of analysis is a decision. The document gave 0.9999; the sentence mean gives something that varies between documents and can be compared.

Does it agree with history?

Scoring all 227 speeches by sentence mean:

Least positive	Roosevelt 1942, 1943, 1944
Most positive	Madison 1815, Adams 1800, Washington 1791
Correlation with year	-0.46

The three most negative addresses in 227 years are the Second World War.

Interpreting sentiment

Ask

- Does sentiment differ by document, or by president?
- Do certain eras appear more negative?
- How do crises influence tone?
- **Do pre-processing decisions distort results?**

Important

- Sentiment $\neq$ emotion
- Lexicons $\neq$ universal
- Results are **relative, not absolute**

If you removed stop words, you removed **not**. A pipeline that discards negation cannot do sentiment analysis.

Topic modelling and sentiment together

Topic modelling

- Detects **what themes** exist
- Unsupervised
- Good for structure

Sentiment analysis

- Detects **tone**
- Lexicon-based or supervised
- Good for affective trends

Together they are powerful for exploring both content and emotion — for example, is one topic discussed more negatively than another?

Hill, “Contextual Analytics”, §5.5 — distinguish what the analysis demonstrates from the interpretation built on it.

Part IV: Live Coding

Live coding

1. Build a document–term matrix from the corpus
2. Run LDA, read the topics, notice what is wrong with them
3. Fix the pre-processing and run it again
4. Sentiment on a whole document — and why it fails
5. Sentiment by sentence, across the corpus
6. Sentiment over time

That's it

The report is due 21 January 2027.

There is a **project clinic in January** — bring your analysis and your questions.

Note

Today's quiz was the last. Nine quizzes, 40% of your mark, done.

Thank you
Questions?



Speaker notes